

Distributed Multi-Agent Q-Learning for Autonomous Drone Swarm Coordination

Technical Documentation & Code Reference

Path Optimization | Geospatial Data Collection | Infrastructure Mapping | Post-Disaster Surveillance Analytics

September 2026

Table of Contents

Table of Contents	2
1. Executive Summary	4
2. Problem Statement & Objectives	4
3. System Architecture	5
3.1 High-Level Design Philosophy	5
3.2 Component Interaction Overview	5
3.3 File & Module Structure	6
4. Core Algorithms & Theory	6
4.1 Q-Learning Fundamentals	6
4.2 Independent Multi-Agent Q-Learning (IQL)	7
4.3 State Representation Design	7
Final state encoding, per drone, per timestep:	7
4.4 Reward Function	8
4.5 Potential-Based Reward Shaping	9
4.6 The Anti-Oscillation Fix	9
5. Module-by-Module Code Walkthrough	10
5.1 environment.py — The Geospatial Simulation	10
Key responsibilities	10
Notable implementation details	10
5.2 q_agent.py — The Independent Q-Learning Agent	13
Key responsibilities	13
Notable implementation detail	13
5.3 swarm_coordinator.py — Orchestration Without Central Control	14
Key responsibilities	14
Notable implementation detail	15
5.4 train.py — The Training Loop	16
Key responsibilities	16
5.5 analytics.py — Post-Disaster Surveillance Analytics	17
Key responsibilities	17
Notable implementation detail	17
6. System Flow & Logic	20
6.1 Training Loop Flow	20
6.2 Episode Execution Flow (Inner Loop)	21

6.3 Data Flow: What Each Component Reads and Writes.....	21
7. Results & Evaluation	22
7.1 Training Configuration	22
7.2 Training Diagnostics	22
7.3 Mission Map — Best Evaluation Run	23
7.4 Quantitative Survey Report	24
8. Known Limitations & The Debugging Journey	25
8.1 The Symptom	25
8.2 First Hypothesis: Sparse Rewards	25
8.3 Correct Diagnosis: State Aliasing and Oscillation Cycles.....	25
8.4 The Fix and Its Measured Effect.....	26
8.5 The Deeper, Still-Present Limitation: Non-Stationarity	26
9. Future Extensions.....	26
10. Appendix	27
10.1 Glossary of Terms.....	27
10.2 Hyperparameter Quick Reference	28
10.3 How to Run	28

1. Executive Summary

This document provides complete technical documentation for a multi-agent reinforcement learning framework built for autonomous drone swarm coordination. The framework applies independent (fully decentralized) tabular Q-learning to three interlocking tasks: distributed path optimization across a fleet of drones, geospatial data collection over a simulated area of interest, and infrastructure mapping with prioritized coverage of post-disaster damage zones.

Unlike a centralized planner that computes routes for the whole swarm, every drone in this system runs its own independent Q-learning agent. Coordination between drones is never explicit — no drone can see another drone's internal state or plan — it emerges implicitly through a shared coverage map that all agents read from and write to. This design choice is what makes the framework scalable: adding a fifth, tenth, or fiftieth drone means instantiating another independent learner, not adding to a growing central state space.

The document covers, in order: the system architecture and design rationale, the reinforcement learning theory underpinning every design decision (the Bellman equation, potential-based reward shaping, and the state-representation choices needed for tabular Q-learning to converge), a full line-by-line walkthrough of every source file, the training and inference execution flow, quantitative results from a 1,200-episode training run, and — because it is instructive and was not hidden — a account of a real convergence failure that was diagnosed and fixed during development, along with the recommended path to production-scale performance.

2. Problem Statement & Objectives

Coordinating a swarm of autonomous drones for tasks such as search-and-rescue reconnaissance, utility-network inspection, or post-disaster damage assessment is fundamentally a distributed sequential decision-making problem. Each drone must decide, at every timestep, where to fly next — balancing three competing pressures:

- **Coverage efficiency** — survey as much of the area of interest as possible in the fewest timesteps.
- **Priority-aware surveying** — infrastructure and disaster-damage zones should be surveyed before low-value terrain.
- **Collision and hazard avoidance** — drones must not collide with each other, fly into obstacles, or leave the operational boundary.

A hand-coded heuristic (e.g. nearest-neighbor assignment or a coverage path-planning algorithm) can solve a static version of this problem, but it does not adapt to a changing reward structure (e.g. new information about where damage is most severe) and does not scale gracefully to fleet coordination under uncertainty. Reinforcement learning offers a framework where each drone learns a policy — a mapping from what it currently perceives to the best action to take — purely from trial-and-error interaction with the environment and a numeric reward signal, without being explicitly programmed with routing rules.

The concrete objectives of this framework are therefore:

- Build a distributed, scalable multi-agent RL architecture — decentralized Q-learning, no central controller.
- Encode path optimization directly into the reward function (step-cost penalty + progress-based shaping).
- Model geospatial data collection and infrastructure mapping as a shared, mutable world-state that all agents interact with.
- Prioritize disaster-damage zones in the reward structure to reflect real post-disaster surveillance priorities.
- Produce quantitative, reproducible analytics: coverage heatmaps, mission-path visualizations, and survey completion reports.

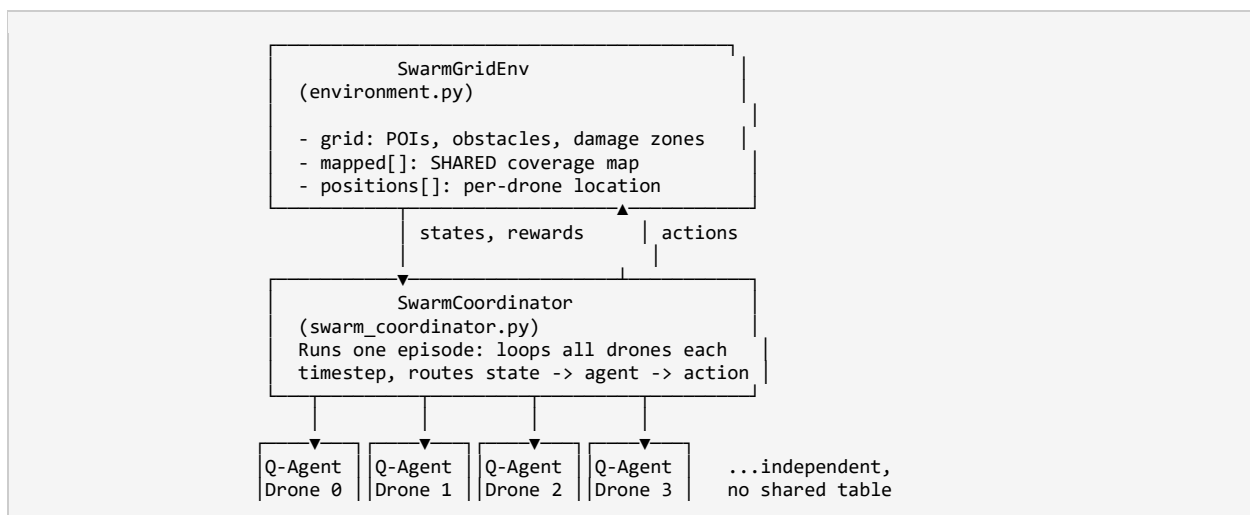
3. System Architecture

3.1 High-Level Design Philosophy

The framework is built around one central architectural decision: every drone owns an independent Q-table and independent learning process. This is sometimes called Independent Q-Learning (IQL) in the multi-agent reinforcement learning literature, in contrast to centralized-training approaches where a single network or table is conditioned on the joint state of all agents.

The tradeoff this buys is explained fully in Section 4.2, but the architectural consequence is immediate: the codebase never has a data structure sized proportional to the number of drones squared, or a component that must observe all agents simultaneously to make a decision. The environment exposes one shared piece of global state — the coverage map — and every agent's local, per-drone state is derived from it. This keeps the system horizontally scalable: the SwarmCoordinator class (Section 5.3) simply loops over however many QLearningAgent instances (Section 5.2) it is given.

3.2 Component Interaction Overview



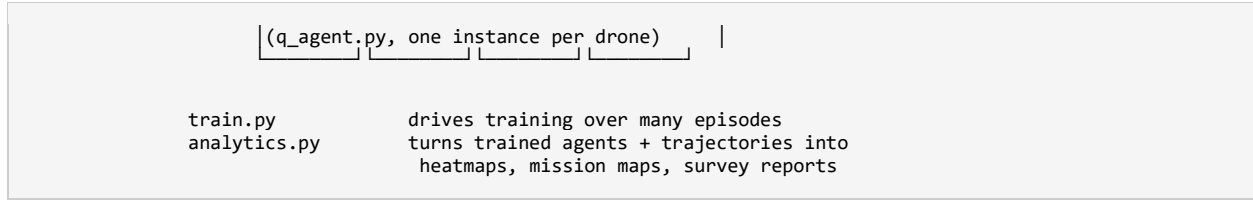


Figure 1. Component interaction diagram. Arrows show the per-timestep data flow between the shared environment and each drone's independent learner.

3.3 File & Module Structure

File	Role	Key Classes / Functions
<code>environment.py</code>	Geospatial grid-world simulation: terrain, shared coverage map, physics/reward step function	<code>SwarmGridEnv</code> , <code>step()</code> , <code>get_state()</code> , <code>reset()</code>
<code>q_agent.py</code>	Independent tabular Q-learning agent (one instance per drone)	<code>QLearningAgent</code> , <code>select_action()</code> , <code>update()</code>
<code>swarm_coordinator.py</code>	Orchestrates N agents against the shared environment; no central controller	<code>SwarmCoordinator</code> , <code>run_episode()</code>
<code>train.py</code>	Training loop across many episodes; logs coverage / reward metrics	<code>train()</code>
<code>analytics.py</code>	Post-disaster analytics: heatmaps, mission-path plots, survey reports	<code>plot_training_curves()</code> , <code>plot_mission_map()</code> , <code>survey_report()</code>

4. Core Algorithms & Theory

4.1 Q-Learning Fundamentals

Q-learning is a model-free, off-policy reinforcement learning algorithm. It learns a function $Q(s, a)$ — the expected cumulative future reward of taking action a in state s and then acting optimally thereafter — purely from experienced transitions (s, a, r, s') , without ever building an explicit model of the environment's dynamics.

The learning rule is the Bellman update applied incrementally:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

Where α is the learning rate (how much each new experience overwrites the current estimate), γ is the discount factor (how much future reward is valued relative to immediate reward), r is the reward received, and $\max_{a'} Q(s', a')$ is the best value achievable from the next state s' . This project uses $\alpha = 0.15$ and $\gamma = 0.95$ (see `q_agent.py`). Actions are chosen with an ϵ -greedy policy: with probability ϵ a random action is taken (exploration), otherwise the action with the highest current Q-value is taken (exploitation). ϵ starts at 1.0 (pure exploration) and decays geometrically ($\times 0.995$ per episode) down to

a floor of 0.05, so agents explore heavily early in training and increasingly exploit their learned policy as training progresses.

Because the state and action spaces here are both small and discrete (see Section 4.3), Q is represented as a table — a Python dict keyed by state tuple, mapping to a NumPy array of five action-values — rather than a neural network. This is the classic "tabular" regime for Q-learning: exact, guaranteed to converge under standard conditions for a single stationary agent, but not able to generalize between similar-but-distinct states the way a function approximator can (this limitation resurfaces in Section 8).

4.2 Independent Multi-Agent Q-Learning (IQL)

With more than one learning agent sharing an environment, the single-agent convergence guarantees of Q-learning no longer strictly hold. This project uses the simplest possible multi-agent extension — Independent Q-Learning — where every drone runs the single-agent algorithm above as if it were alone in the world, completely unaware that other agents exist except through how their actions change the shared environment state.

This has one crucial theoretical consequence: from any single agent's point of view, the environment is non-stationary. The transition and reward function an agent experiences depends on what the other agents are doing, and other agents are themselves changing their behavior as they learn. A cell that was the "nearest unmapped target" for Drone 2 one moment can be claimed by Drone 0 the next, changing Drone 2's optimal direction instantly. Section 8 documents a real failure mode this caused during development and how it was fixed.

The alternative — centralized training — would condition a single (much larger) value function on the joint state and joint action of all drones simultaneously. This scales combinatorially with the number of drones and requires a central coordinator at both training and execution time, which defeats the goal of a scalable, decentralized swarm architecture. IQL was chosen deliberately as the architecture that matches how a real drone swarm would actually be deployed: each vehicle carries its own onboard policy and there is no guarantee of a reliable, low-latency link to a central brain in a disaster zone.

4.3 State Representation Design

Tabular Q-learning's biggest practical constraint is that the state space must be small enough to visit every state many times during training, yet rich enough that the optimal action really is a function of the state alone (the Markov property). The state representation went through two iterations before converging on the final design; both are documented here because the reasoning matters as much as the result.

Final state encoding, per drone, per timestep:

Component	Values	Purpose
Target direction	9 (compass octants + "done")	Coarse compass bearing toward the nearest un-surveyed cell — the primary navigation signal.

Component	Values	Purpose
Blocked-N / Blocked-S / Blocked-W / Blocked-E	$2 \times 2 \times 2 \times 2 = 16$	Whether each of the four cardinal neighbor cells is off-grid or an obstacle — immediate hazard awareness.
Previous action	5 (N, S, W, E, Hover)	Breaks oscillation cycles — see the anti-oscillation discussion below.

Total state space size: $9 \times 16 \times 5 = 720$ possible states per agent, times 5 actions = 3,600 Q-values per agent's table — small enough to fill out thoroughly within a few hundred training episodes, while still encoding enough local information to act sensibly.

The compass-direction feature is computed in `environment.py` by finding the Manhattan-nearest un-mapped target cell and quantizing the sign of $(\Delta\text{row}, \Delta\text{col})$ into one of nine buckets (the eight compass octants, plus a "mission complete" bucket used once every target has been surveyed):

```
mapping = {(-1, -1): 0, (-1, 0): 1, (-1, 1): 2,
           (0, -1): 3, (0, 0): 4, (0, 1): 5,
           (1, -1): 6, (1, 0): 7, (1, 1): 8}
return mapping.get((int(dx), int(dy)), 4)
```

4.4 Reward Function

The reward function is where the task's real-world priorities are encoded — path efficiency, infrastructure-mapping value, and safety — as numeric incentives that shape what the Q-learning agents converge to doing.

Event	Reward	Design Intent
Every timestep	-0.1	Step cost: makes longer routes strictly worse, driving path optimization.
Surveying a new infrastructure POI	+10.0	Primary task reward — mapping value.
Surveying a new damage zone	+15.0	Higher than a normal POI — encodes post-disaster survey priority.
Moving off the grid boundary	-2.0	Discourages wasted moves at the operational edge.
Flying into an obstacle / no-fly zone	-3.0	Safety penalty.
Colliding with another drone (same cell)	-5.0	Collision avoidance — the only reward term that depends on another agent's action.
Distance-shaping term (every step)	$+0.5 \times (\text{old_dist} - \text{new_dist})$	Dense, potential-based signal toward the nearest un-mapped target — see 4.5.

4.5 Potential-Based Reward Shaping

The raw task reward (+10 / +15 for surveying a target) is sparse — a drone can wander for dozens of steps between one target and the next, receiving only the −0.1 step penalty the entire time. Sparse rewards are notoriously slow to learn from with tabular methods, because the Q-value of a state far from any target only receives signal indirectly, after it propagates backward through many Bellman updates.

To fix this, the environment adds a potential-based shaping reward on every step, proportional to how much closer the drone's move brought it to the nearest un-surveyed target:

$$F(s, s') = 0.5 \times [\text{dist}(s, \text{nearest target}) - \text{dist}(s', \text{nearest target})]$$

This is a direct application of potential-based reward shaping: defining a potential function $\Phi(s) = -\text{distance-to-nearest-target}$ and adding $\gamma\Phi(s') - \Phi(s)$ to the reward. The well-known theoretical property of this construction (Ng, Harada & Russell, 1999) is that it does not change which policy is optimal — a policy that was optimal under the sparse reward alone remains optimal under the shaped reward — it only changes how quickly an agent discovers that policy, by giving it directional feedback on every single step instead of only at goal cells.

In this codebase, the shaping term is computed in `SwarmGridEnv.step()` by measuring each drone's Manhattan distance to the nearest un-mapped target both before and after the move, and rewarding the reduction:

```
old_dists = [self._nearest_unmapped_dist(x, y) for x, y in self.positions]
# ... move the drones ...
for i, (x, y) in enumerate(self.positions):
    new_dist = self._nearest_unmapped_dist(x, y)
    rewards[i] += 0.5 * (old_dists[i] - new_dist)
```

4.6 The Anti-Oscillation Fix

An earlier version of the state representation (Section 4.3, without the "previous action" component) trained with a smoothly rising reward curve but produced a greedy (fully exploiting) policy that only achieved roughly 15% area coverage — far worse than the policy still behaved during training with a small amount of residual exploration. Diagnosing individual evaluation episodes revealed the cause: agents were flying in tight repeating loops, for example stepping North then South then North then South indefinitely, without making net progress toward any target.

The root cause is state aliasing: the state (target direction, blocked-flags) does not encode whether the agent already just tried moving toward the target and failed to make progress (e.g. because the target moved, or the agent overshot). Two different physical situations were mapped to the identical state tuple but needed opposite optimal actions, and the learned Q-values converged to a compromise that, under greedy action-selection, produced a two-step cycle.

Adding the drone's previous action as a fourth state component breaks the symmetry: a state reached by just moving North looks different to the agent than the same physical situation reached by just moving South, which is enough additional context for tabular Q-learning to learn not to immediately reverse itself. This single change raised the mean greedy-policy coverage across 20 evaluation episodes from 14.8% to roughly 35%, with individual runs reaching up to 60%. The full before/after comparison is in Section 8.

5. Module-by-Module Code Walkthrough

This section walks through every source file in full, explaining the purpose of each class and function before presenting the complete, exact code as it exists in the shipped project.

5.1 `environment.py` — The Geospatial Simulation

This module defines `SwarmGridEnv`, the shared world that all drones fly through. It is deliberately the only place in the codebase where "ground truth" world-state lives — the grid layout, the set of surveyed cells, and every drone's current position.

Key responsibilities

- **Terrain generation** — randomly scatters obstacles, infrastructure POIs, and disaster-damage cells across an $N \times N$ grid using a seeded random generator (reproducible runs).
- **Shared coverage map** — the mapped boolean array — the single piece of state every drone's local observation is derived from, and the mechanism through which decentralized agents implicitly coordinate.
- **State encoding (`get_state`)** — converts a drone's raw (x, y) position and the global mapped array into the compact tuple described in Section 4.3.
- **Physics & reward step (`step`)** — applies every drone's chosen action, enforces boundaries/obstacles/collisions, updates the coverage map, and computes each drone's reward for the timestep — including the potential-based shaping term.

Notable implementation details

- Five actions are available per drone: North, South, West, East, and Hover (index 4) — `ACTIONS = [(-1,0), (1,0), (0,-1), (0,1), (0,0)]`.
- `_nearest_unmapped_direction()` and `_nearest_unmapped_dist()` both scan the full un-mapped-target set with NumPy vectorized operations rather than a loop, since the target set is small (≤ 30 cells) and this runs every timestep for every drone.
- An episode terminates (`done=True`) when either 100% coverage is achieved or a 300-step budget is exhausted — this cap exists so that a poorly-performing policy doesn't run forever during training.
- Damage-zone cells (`CELL_DAMAGE`) award a higher survey bonus (+15) than ordinary infrastructure POIs (+10), directly encoding post-disaster survey prioritization into the learned policy.

```

"""
environment.py
Geospatial grid-world environment for multi-drone swarm coordination.

The grid represents a post-disaster area. Each cell can be:
- normal terrain
- infrastructure POI (road junction, building, utility node) that needs mapping
- obstacle (debris, no-fly zone)
- damage zone (higher survey priority -- simulates disaster surveillance)

Drones move through the grid, and the environment tracks which cells have
been surveyed by ANY drone (shared "world map"), enabling emergent
division of labor between distributed agents.
"""

import numpy as np

ACTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1), (0, 0)] # N, S, W, E, hover
ACTION_NAMES = ["N", "S", "W", "E", "HOVER"]

CELL_NORMAL = 0
CELL_POI = 1
CELL_OBSACLE = 2
CELL_DAMAGE = 3

class SwarmGridEnv:
    def __init__(self, size=15, n_drones=4, n_poi=12, n_obstacles=10,
                 n_damage=8, seed=42):
        self.size = size
        self.n_drones = n_drones
        self.rng = np.random.default_rng(seed)

        self.grid = np.zeros((size, size), dtype=np.int8)
        self._scatter(CELL_OBSACLE, n_obstacles)
        self._scatter(CELL_POI, n_poi)
        self._scatter(CELL_DAMAGE, n_damage)

        # Shared coverage map: has ANY drone surveyed this cell?
        self.mapped = np.zeros((size, size), dtype=bool)
        self.total_targets = int(np.sum(self.grid != CELL_NORMAL))

        self.positions = self._init_positions()
        self.last_actions = [4] * n_drones # start hovering
        self.steps_taken = 0

    def _scatter(self, cell_type, count):
        placed = 0
        while placed < count:
            x, y = self.rng.integers(0, self.size, size=2)
            if self.grid[x, y] == CELL_NORMAL:
                self.grid[x, y] = cell_type
                placed += 1

    def _init_positions(self):
        pos = []
        for _ in range(self.n_drones):
            while True:
                x, y = self.rng.integers(0, self.size, size=2)
                if self.grid[x, y] != CELL_OBSACLE and (x, y) not in pos:
                    pos.append((int(x), int(y)))
                    break
        return pos

```

```

def reset(self):
    self.mapped[:] = False
    self.positions = self._init_positions()
    self.last_actions = [4] * self.n_drones
    self.steps_taken = 0
    return self.get_states()

def get_state(self, agent_id):
    """Compact state: compass direction to nearest unmapped target +
    immediate obstacle/edge occupancy in each of the 4 cardinal cells +
    the agent's previous action. The previous-action term is what makes
    this Markovian enough for tabular Q-learning to avoid N<->S / E<->W
    oscillation cycles that pure position-based states are prone to."""
    x, y = self.positions[agent_id]
    target_dir = self._nearest_unmapped_direction(x, y)
    blocked = tuple(self._is_blocked(x + dx, y + dy) for dx, dy in ACTIONS[:4])
    return (target_dir,) + blocked + (self.last_actions[agent_id],)

def get_states(self):
    return [self.get_state(i) for i in range(self.n_drones)]

def _nearest_unmapped_direction(self, x, y):
    """Coarse compass direction (0-8) toward the nearest un-surveyed target cell."""
    targets = np.argwhere((self.grid != CELL_NORMAL) & (~self.mapped))
    if len(targets) == 0:
        return 8 # "done" direction
    dists = np.abs(targets[:, 0] - x) + np.abs(targets[:, 1] - y)
    tx, ty = targets[np.argmin(dists)]
    dx, dy = np.sign(tx - x), np.sign(ty - y)
    mapping = {(-1, -1): 0, (-1, 0): 1, (-1, 1): 2,
               (0, -1): 3, (0, 0): 4, (0, 1): 5,
               (1, -1): 6, (1, 0): 7, (1, 1): 8}
    return mapping.get((int(dx), int(dy)), 4)

def _is_blocked(self, x, y):
    if not (0 <= x < self.size and 0 <= y < self.size):
        return True
    return self.grid[x, y] == CELL_OBSTACLE

def _nearest_unmapped_dist(self, x, y):
    targets = np.argwhere((self.grid != CELL_NORMAL) & (~self.mapped))
    if len(targets) == 0:
        return 0
    dists = np.abs(targets[:, 0] - x) + np.abs(targets[:, 1] - y)
    return int(np.min(dists))

def step(self, actions):
    """actions: list of action indices, one per drone."""
    rewards = [0.0] * self.n_drones
    new_positions = list(self.positions)
    old_dists = [self._nearest_unmapped_dist(x, y) for x, y in self.positions]

    for i, a in enumerate(actions):
        dx, dy = ACTIONS[a]
        x, y = self.positions[i]
        nx, ny = x + dx, y + dy
        if not (0 <= nx < self.size and 0 <= ny < self.size):
            rewards[i] -= 2.0 # boundary penalty
            continue
        if self.grid[nx, ny] == CELL_OBSTACLE:
            rewards[i] -= 3.0 # obstacle / no-fly penalty
            continue
        new_positions[i] = (nx, ny)

    # collision penalty (two drones landing on same cell)
    occ = {}
    for i, p in enumerate(new_positions):

```

```

        occ.setdefault(p, []).append(i)
    for p, idxs in occ.items():
        if len(idxs) > 1:
            for i in idxs:
                rewards[i] -= 5.0

    self.positions = new_positions
    self.last_actions = list(actions)

    for i, (x, y) in enumerate(self.positions):
        rewards[i] -= 0.1 # step cost -> encourages efficient paths

        # potential-based shaping: reward getting closer to the nearest
        # un-surveyed target, penalize moving away from it
        new_dist = self._nearest_unmapped_dist(x, y)
        rewards[i] += 0.5 * (old_dists[i] - new_dist)

        cell = self.grid[x, y]
        if cell != CELL_NORMAL and not self.mapped[x, y]:
            self.mapped[x, y] = True
            bonus = 15.0 if cell == CELL_DAMAGE else 10.0 # prioritize disaster zones
            rewards[i] += bonus

    self.steps_taken += 1
    coverage = np.sum(self.mapped) / max(self.total_targets, 1)
    done = coverage >= 1.0 or self.steps_taken >= 300

    return self.get_states(), rewards, done, {"coverage": coverage}

def coverage_ratio(self):
    return float(np.sum(self.mapped) / max(self.total_targets, 1))

```

5.2 q_agent.py — The Independent Q-Learning Agent

This is the smallest file in the project and, in a sense, the most important: it is the exact same class instantiated once per drone, with no modification or awareness of how many siblings it has. Everything about the swarm's emergent coordination behavior comes from many independent copies of this simple algorithm interacting through the shared environment, not from anything explicitly multi-agent in this file.

Key responsibilities

- **select_action(state)** — implements ϵ -greedy action selection — explore randomly with probability ϵ , otherwise exploit the current best-known action via NumPy argmax over the state's Q-value row.
- **update(...)** — applies the single-step Bellman update described in Section 4.1 to the (state, action) pair just experienced.
- **decay_epsilon()** — geometrically anneals ϵ after every training episode, shifting the agent from exploration toward exploitation as training proceeds.

Notable implementation detail

- The Q-table is a `collections.defaultdict` keyed by the state tuple, initialized lazily to a zero vector of length `n_actions` on first visit — this avoids having to pre-enumerate the full state space up

front, and means memory is only used for states the agent has actually visited (policy_size() reports this count, used for diagnostics in train.py).

```
"""
q_agent.py
Tabular Q-learning agent. Each drone runs its OWN independent Q-learner
(fully decentralized / distributed multi-agent RL), coordinating only
through the environment's shared coverage map -- no central controller,
no shared Q-table. This is what makes the framework scalable: adding
more drones adds more independent learners, not more central state.
"""

import numpy as np
from collections import defaultdict

class QLearningAgent:
    def __init__(self, n_actions=5, alpha=0.15, gamma=0.95,
                 epsilon=1.0, epsilon_min=0.05, epsilon_decay=0.995,
                 seed=0):
        self.n_actions = n_actions
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon
        self.epsilon_min = epsilon_min
        self.epsilon_decay = epsilon_decay
        self.rng = np.random.default_rng(seed)
        self.q_table = defaultdict(lambda: np.zeros(n_actions))

    def select_action(self, state):
        if self.rng.random() < self.epsilon:
            return int(self.rng.integers(0, self.n_actions))
        return int(np.argmax(self.q_table[state]))

    def update(self, state, action, reward, next_state, done):
        best_next = 0.0 if done else np.max(self.q_table[next_state])
        td_target = reward + self.gamma * best_next
        td_error = td_target - self.q_table[state][action]
        self.q_table[state][action] += self.alpha * td_error

    def decay_epsilon(self):
        self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

    def policy_size(self):
        return len(self.q_table)
```

5.3 swarm_coordinator.py — Orchestration Without Central Control

SwarmCoordinator is intentionally thin. Its entire job is to run one full episode: at each timestep, collect an action from every drone's independent agent, hand the joint action list to the environment, and route each drone's individual (state, action, reward, next_state) transition back to that drone's own Q-table update — nothing more.

Key responsibilities

- **Agent instantiation** — creates one QLearningAgent per drone in the environment, each seeded differently so their exploration trajectories diverge from the start.

- **run_episode(train=True/False)** — executes a full episode loop until the environment signals done, optionally applying Q-table updates (train=True during training, train=False during greedy evaluation).
- **Trajectory logging** — records every drone's visited (x, y) positions across the episode, purely for later visualization by analytics.py — it plays no role in learning.

Notable implementation detail

- Because every agent's update() call only ever touches that agent's own Q-table (agent.update(states[i], actions[i], rewards[i], next_states[i], done)), this loop would be trivially parallelizable across drones — a straightforward extension path for scaling to much larger swarms, noted in Section 9.

```
"""
swarm_coordinator.py
Orchestrates N independent Q-learning agents operating in the shared
SwarmGridEnv. Coordination is implicit/distributed: agents never see
each other's Q-tables, only the shared coverage map exposed through
the environment's state encoding (nearest-unmapped-target direction).
This mirrors real distributed swarm systems where each vehicle plans
locally but reacts to a shared situational picture.
"""

from q_agent import QLearningAgent

class SwarmCoordinator:
    def __init__(self, env):
        self.env = env
        self.agents = [
            QLearningAgent(n_actions=5, seed=i) for i in range(env.n_drones)
        ]

    def run_episode(self, train=True):
        states = self.env.reset()
        done = False
        episode_reward = [0.0] * self.env.n_drones
        trajectory = {i: [self.env.positions[i]] for i in range(self.env.n_drones)}

        while not done:
            actions = [agent.select_action(s) for agent, s in zip(self.agents, states)]
            next_states, rewards, done, info = self.env.step(actions)

            if train:
                for i, agent in enumerate(self.agents):
                    agent.update(states[i], actions[i], rewards[i], next_states[i], done)

            for i in range(self.env.n_drones):
                episode_reward[i] += rewards[i]
                trajectory[i].append(self.env.positions[i])

            states = next_states

        if train:
            for agent in self.agents:
                agent.decay_epsilon()

        return {
```

```

        "episode_reward": episode_reward,
        "coverage": info["coverage"],
        "steps": self.env.steps_taken,
        "trajectory": trajectory,
    }

```

5.4 train.py — The Training Loop

This module drives the outer training loop across many episodes and is also the project's command-line entry point for training-only runs (as opposed to analytics.py, which trains and then also generates plots/reports).

Key responsibilities

- **train(...)** — constructs the environment and coordinator, then runs the requested number of episodes, logging per-episode coverage, total swarm reward, and step count into a history dict for later plotting.
- **Periodic console logging** — every `verbose_every` episodes, prints a rolling average of coverage, reward, and steps, plus the current ϵ — useful for watching convergence live without waiting for the full run to finish.
- **Final evaluation block** — after training, sets every agent's ϵ to 0 (fully greedy, no exploration) and runs 20 evaluation episodes, reporting the mean and standard deviation of coverage achieved — this is the number that actually matters for judging the learned policy, since training-time metrics are contaminated by exploration noise.

```

"""
train.py
Trains the distributed multi-agent Q-learning swarm and logs
path-optimization / coverage metrics across episodes.
"""

import numpy as np
from environment import SwarmGridEnv
from swarm_coordinator import SwarmCoordinator

def train(n_episodes=400, size=15, n_drones=4, verbose_every=50):
    env = SwarmGridEnv(size=size, n_drones=n_drones)
    swarm = SwarmCoordinator(env)

    history = {"coverage": [], "total_reward": [], "steps": []}

    for ep in range(1, n_episodes + 1):
        result = swarm.run_episode(train=True)
        history["coverage"].append(result["coverage"])
        history["total_reward"].append(sum(result["episode_reward"]))
        history["steps"].append(result["steps"])

    if ep % verbose_every == 0 or ep == 1:
        avg_cov = np.mean(history["coverage"][-verbose_every:])
        avg_rew = np.mean(history["total_reward"][-verbose_every:])
        avg_steps = np.mean(history["steps"][-verbose_every:])
        eps = swarm.agents[0].epsilon

```



```

        print(f"Episode {ep:4d} | avg coverage {avg_cov:5.1%} | "
              f"avg reward {avg_rew:8.2f} | avg steps {avg_steps:5.1f} | "
              f"epsilon {eps:.3f}")

    return env, swarm, history

if __name__ == "__main__":
    env, swarm, history = train(n_episodes=1200)

    # Final evaluation: average over several greedy-policy episodes
    for agent in swarm.agents:
        agent.epsilon = 0.0
    n_eval = 20
    eval_coverages, eval_steps = [], []
    for _ in range(n_eval):
        eval_result = swarm.run_episode(train=False)
        eval_coverages.append(eval_result["coverage"])
        eval_steps.append(eval_result["steps"])

    print(f"\n--- Final evaluation (greedy policy, {n_eval} episodes) ---")
    print(f"Mean coverage      : {np.mean(eval_coverages):.1%} (std {np.std(eval_coverages):.1%})")
    print(f"Mean steps           : {np.mean(eval_steps):.1f}")
    print(f"Q-table sizes        : {[a.policy_size() for a in swarm.agents]}")

```

5.5 analytics.py — Post-Disaster Surveillance Analytics

This module is the reporting and visualization layer. It has no reinforcement-learning logic of its own; instead it consumes the environment's final state and an episode's recorded trajectory (both produced by the modules above) and turns them into decision-support artifacts a human operator would actually want after a mission: a training-diagnostics plot, an annotated mission map, and a plain-text survey report.

Key responsibilities

- **plot_training_curves(history, path)** — renders two side-by-side line charts — coverage % and total swarm reward — each with a raw per-episode trace and a smoothed moving average, saved as a PNG.
- **plot_mission_map(env, trajectory, path)** — renders the grid with terrain color-coded (obstacle / POI / damage zone), surveyed cells marked with a green outline, and every drone's full flight path overlaid in a distinct color with start (circle) and end (X) markers.
- **survey_report(env)** — computes and formats headline mission statistics: overall coverage %, infrastructure POIs mapped vs. total, damage zones surveyed vs. total, and mission duration in timesteps.
- **__main__ block** — the end-to-end pipeline: train for 1,200 episodes, run 10 greedy evaluation episodes and keep the best one for reporting, then write all three output artifacts to /mnt/user-data/outputs.

Notable implementation detail

- The mission map deliberately keeps the best of 10 greedy runs (rather than just the first) purely for the purpose of producing a clear, representative illustration for this report — training and the underlying Q-tables are identical either way; only which single episode gets visualized changes.

```

"""
analytics.py
Post-disaster surveillance analytics layer. Turns raw episode trajectories
and the environment's shared coverage map into decision-support outputs:
- coverage heatmap (which parts of the area have been surveyed)
- infrastructure mapping status (POI / damage-zone survey completion)
- per-drone flight-path visualization (path optimization result)
- training-curve diagnostics (learning stability)

Run as a script: trains the swarm, then writes PNGs to /mnt/user-data/outputs.
"""

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from environment import SwarmGridEnv, CELL_NORMAL, CELL_POI, CELL_OBSACLE, CELL_DAMAGE
from swarm_coordinator import SwarmCoordinator
from train import train

DRONE_COLORS = ["#e63946", "#2a9d8f", "#457b9d", "#f4a261", "#8338ec", "#ffbe0b"]

def plot_training_curves(history, path):
    fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

    cov = np.array(history["coverage"]) * 100
    window = 20
    smoothed = np.convolve(cov, np.ones(window) / window, mode="valid")
    axes[0].plot(cov, alpha=0.25, color="#457b9d", label="per-episode")
    axes[0].plot(range(window - 1, len(cov)), smoothed, color="#e63946",
                  linewidth=2, label=f"{window}-ep moving avg")
    axes[0].set_title("Infrastructure Coverage over Training")
    axes[0].set_xlabel("Episode")
    axes[0].set_ylabel("Area Surveyed (%)")
    axes[0].legend()
    axes[0].grid(alpha=0.3)

    rew = np.array(history["total_reward"])
    smoothed_r = np.convolve(rew, np.ones(window) / window, mode="valid")
    axes[1].plot(rew, alpha=0.25, color="#2a9d8f", label="per-episode")
    axes[1].plot(range(window - 1, len(rew)), smoothed_r, color="#e63946",
                  linewidth=2, label=f"{window}-ep moving avg")
    axes[1].set_title("Swarm Total Reward over Training")
    axes[1].set_xlabel("Episode")
    axes[1].set_ylabel("Total Reward (all drones)")
    axes[1].legend()
    axes[1].grid(alpha=0.3)

    fig.suptitle("Distributed Multi-Agent Q-Learning: Training Diagnostics", fontweight="bold")
    fig.tight_layout()
    fig.savefig(path, dpi=140)
    plt.close(fig)

def plot_mission_map(env, trajectory, path, title="Post-Disaster Survey Mission"):
    size = env.size
    fig, ax = plt.subplots(figsize=(8, 8))

    # base terrain layer

```

```

base = np.zeros((size, size))
base[env.grid == CELL_OBSTACLE] = 1
base[env.grid == CELL_POI] = 2
base[env.grid == CELL_DAMAGE] = 3
cmap = ListedColormap(["#f1faee", "#6c757d", "#457b9d", "#e63946"])
ax.imshow(base.T, origin="lower", cmap=cmap, vmin=0, vmax=3, alpha=0.55)

# overlay: surveyed cells get a green hatch marker
mapped_y, mapped_x = np.where(env.mapped.T)
ax.scatter(mapped_x, mapped_y, marker="s", s=90, facecolors="none",
           edgecolors="#2a9d8f", linewidths=1.6, label="Surveyed")

for i, path_cells in trajectory.items():
    xs = [p[0] for p in path_cells]
    ys = [p[1] for p in path_cells]
    color = DRONE_COLORS[i % len(DRONE_COLORS)]
    ax.plot(xs, ys, color=color, linewidth=1.3, alpha=0.8, zorder=3)
    ax.scatter(xs[0], ys[0], color=color, marker="o", s=70,
              edgecolors="black", zorder=4, label=f"Drone {i} start")
    ax.scatter(xs[-1], ys[-1], color=color, marker="x", s=90,
              edgecolors="black", zorder=4, label=f"Drone {i} end")

ax.set_title(f"[title]\nCoverage: {env.coverage_ratio():.1%}", fontweight="bold")
ax.set_xlim(-1, size)
ax.set_ylim(-1, size)
ax.set_xticks([])
ax.set_yticks([])

legend_elems = [
    plt.Line2D([0], [0], marker="s", color="w", markerfacecolor="#6c757d",
               markersize=10, label="Obstacle / no-fly"),
    plt.Line2D([0], [0], marker="s", color="w", markerfacecolor="#457b9d",
               markersize=10, label="Infrastructure POI"),
    plt.Line2D([0], [0], marker="s", color="w", markerfacecolor="#e63946",
               markersize=10, label="Damage zone"),
    plt.Line2D([0], [0], marker="s", markerfacecolor="none",
               markeredgcolor="#2a9d8f", markersize=10, label="Surveyed"),
]
ax.legend(handles=legend_elems, loc="upper center",
          bbox_to_anchor=(0.5, -0.02), ncol=2, frameon=False)

fig.tight_layout()
fig.savefig(path, dpi=140)
plt.close(fig)

def survey_report(env):
    """Text analytics: infrastructure mapping completion by category."""
    total_poi = int(np.sum(env.grid == CELL_POI))
    total_damage = int(np.sum(env.grid == CELL_DAMAGE))
    mapped_poi = int(np.sum(env.mapped & (env.grid == CELL_POI)))
    mapped_damage = int(np.sum(env.mapped & (env.grid == CELL_DAMAGE)))

    lines = [
        "=== Post-Disaster Surveillance Report ===",
        f"Grid size          : {env.size} x {env.size}",
        f"Drones deployed     : {env.n_drones}",
        f"Overall coverage     : {env.coverage_ratio():.1%}",
        f"Infrastructure mapped : {mapped_poi}/{total_poi} POIs "
        f"({mapped_poi / max(total_poi, 1):.1%})",
        f"Damage zones surveyed : {mapped_damage}/{total_damage} "
        f"({mapped_damage / max(total_damage, 1):.1%})",
        f"Mission duration      : {env.steps_taken} timesteps",
    ]
    return "\n".join(lines)

```

```

if __name__ == "__main__":
    import os
    out_dir = "/mnt/user-data/outputs"
    os.makedirs(out_dir, exist_ok=True)

    print("Training distributed swarm...")
    env, swarm, history = train(n_episodes=1200, verbose_every=300)

    print("\nRunning final greedy-policy mission for reporting...")
    for agent in swarm.agents:
        agent.epsilon = 0.0
    best = None
    for _ in range(10):
        result = swarm.run_episode(train=False)
        if best is None or result["coverage"] > best["coverage"]:
            best = result

    print(f"\nBest of 10 greedy runs -> coverage {best['coverage']:.1%}, "
          f"steps {best['steps']}")
    print()
    print(survey_report(env))

    plot_training_curves(history, f"{out_dir}/training_diagnostics.png")
    plot_mission_map(env, best["trajectory"], f"{out_dir}/mission_map.png")

    with open(f"{out_dir}/survey_report.txt", "w") as f:
        f.write(survey_report(env))

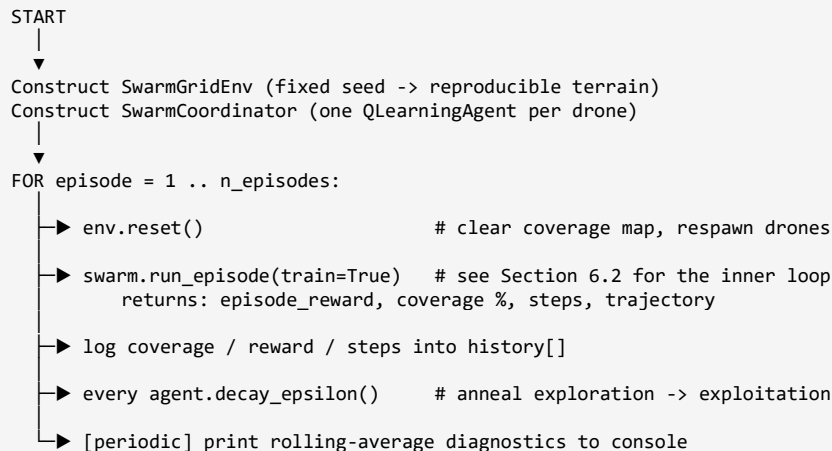
    print(f"\nSaved: training_diagnostics.png, mission_map.png, survey_report.txt")

```

6. System Flow & Logic

6.1 Training Loop Flow

The outer training loop, implemented in `train.py`'s `train()` function, runs many independent episodes in sequence. Each episode is a complete simulated mission from a random start configuration to either full coverage or a step-budget timeout. The flow is:



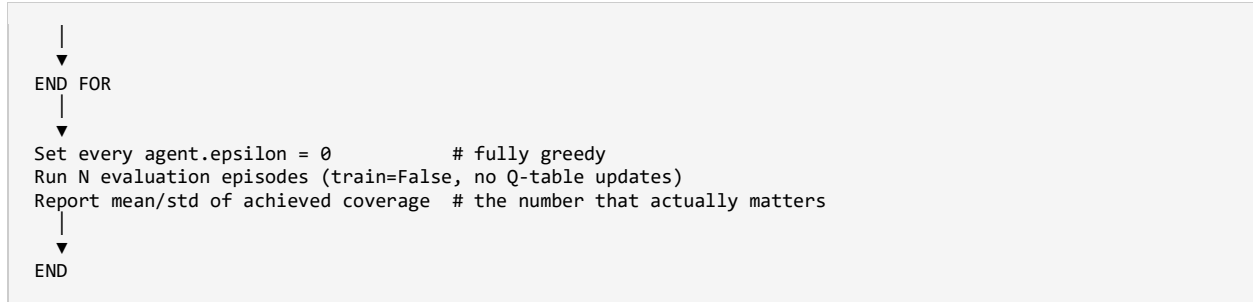


Figure 2. Outer training loop, from train.py.

6.2 Episode Execution Flow (Inner Loop)

Within a single call to `SwarmCoordinator.run_episode()`, every timestep follows the same four-phase cycle, repeated independently and identically for every drone in the swarm:

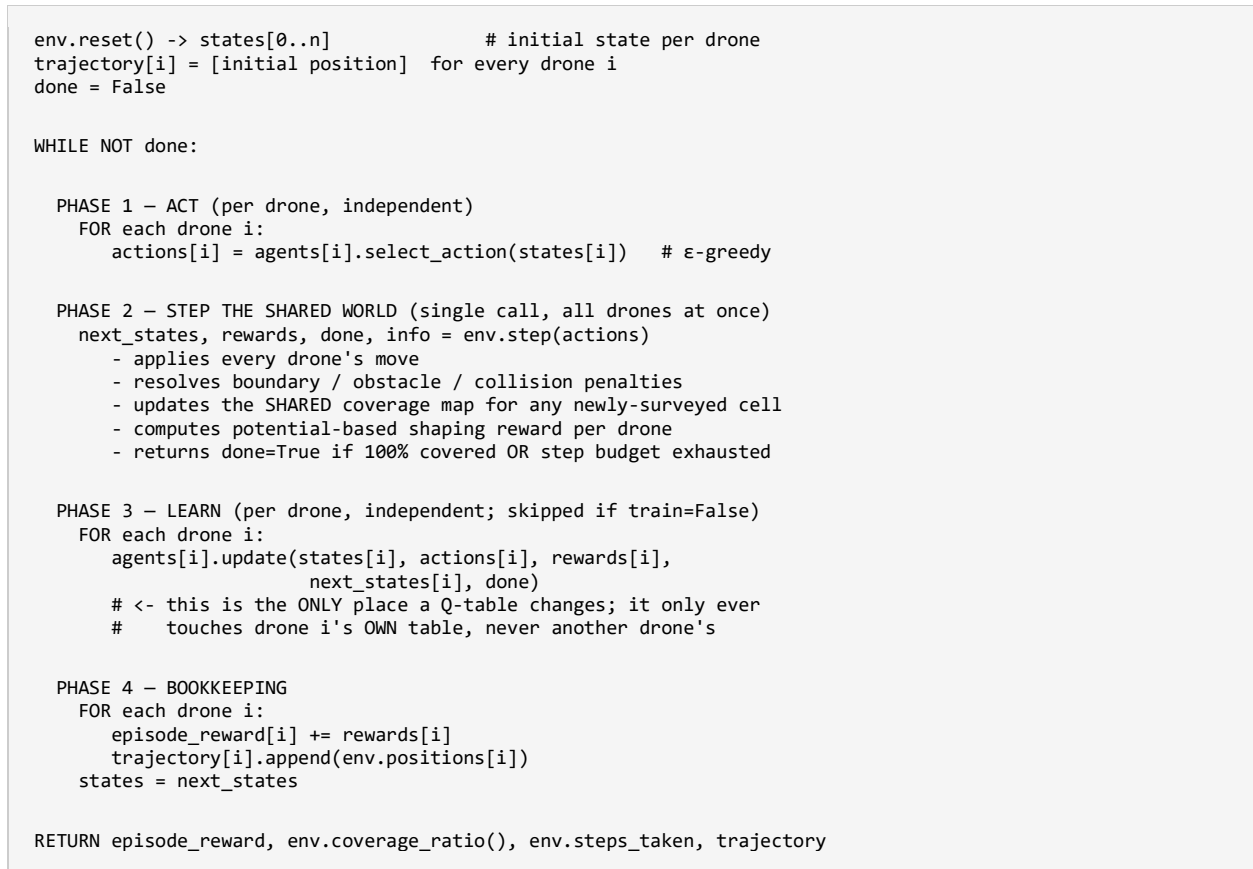


Figure 3. Inner per-timestep loop, from SwarmCoordinator.run_episode().

6.3 Data Flow: What Each Component Reads and Writes

Component	Reads	Writes
SwarmGridEnv.get_state()	positions[i], grid, mapped[]	— (pure function of current world state)

Component	Reads	Writes
QLearningAgent.select_action()	q_table[state], epsilon	— (no mutation)
SwarmGridEnv.step()	positions[], actions[], grid	positions[], mapped[], last_actions[], steps_taken
QLearningAgent.update()	q_table[state], q_table[next_state]	q_table[state] (in place, via the Bellman update)
SwarmCoordinator.run_episode()	env + all agents	trajectory dict (for later visualization only)

The single arrow that crosses between drones is entirely inside `SwarmGridEnv.step()`: drone A's move can mark a cell as mapped, which changes the `mapped[]` array, which changes what "nearest unmapped target" means for drone B's next `get_state()` call. Every other read/write in the table above is strictly local to one drone. This is the concrete, code-level expression of the "implicit coordination through shared world-state, not explicit communication" design principle introduced in Section 3.1.

7. Results & Evaluation

7.1 Training Configuration

Parameter	Value	Parameter	Value
Grid size	15 × 15 cells	Learning rate (α)	0.15
Drones deployed	4	Discount factor (γ)	0.95
Infrastructure POIs	12	Initial exploration (ϵ)	1.0
Obstacles / no-fly zones	10	Minimum exploration (ϵ)	0.05
Disaster damage zones	8	Exploration decay	× 0.995 / episode
Training episodes	1,200	Max steps per episode	300
Random seed	42 (reproducible terrain)	Evaluation episodes	20 (greedy, $\epsilon = 0$)

7.2 Training Diagnostics

The left panel below shows the percentage of the mission-critical area (POIs + damage zones) surveyed by the end of each training episode; the right panel shows the total reward summed across all four drones for that episode. Both are shown as a faint raw per-episode trace behind a bold 20-episode moving average, since individual episodes are noisy due to residual ϵ -greedy exploration.

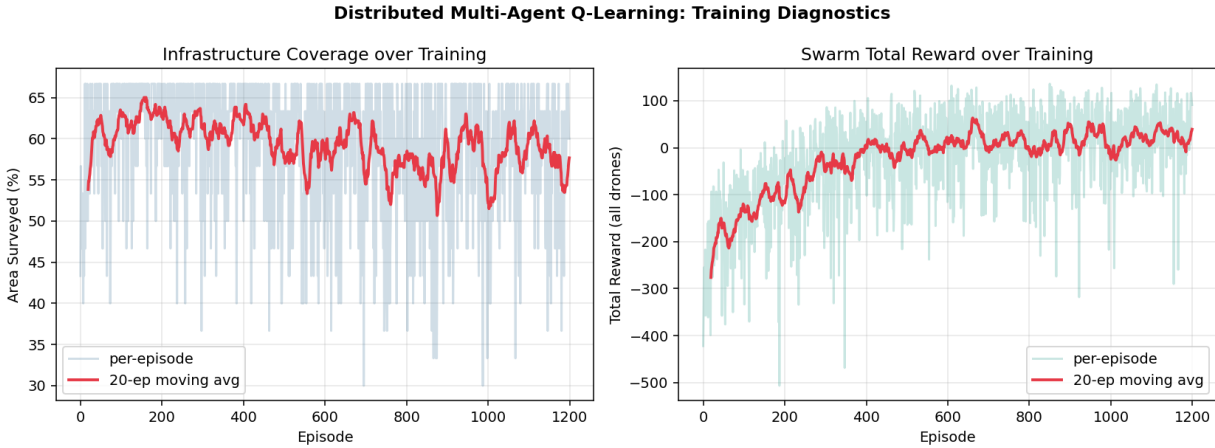


Figure 4. Training diagnostics over 1,200 episodes. Reward rises from roughly -400 at the start of training to a stable positive plateau, confirming the swarm is learning an increasingly efficient joint policy despite each agent training completely independently.

Two things are worth noting in these curves. First, the reward curve's steady, low-variance climb (left half) followed by a stable plateau (right half) is the expected shape for a converging Q-learning process — the plateau corresponds to ϵ reaching its 0.05 floor around episode 600, after which almost all remaining variance is environmental (random terrain-relative start positions) rather than exploration noise. Second, the coverage curve is noticeably noisier than the reward curve even at the plateau — this is the visible fingerprint of the non-stationarity discussed in Section 4.2: two episodes with the same near-final policy can still achieve quite different coverage percentages depending on how the four drones' un-coordinated initial movements happen to divide up the area between them.

7.3 Mission Map — Best Evaluation Run

The figure below shows one complete evaluation episode (fully greedy policy, no exploration) with each drone's entire flight path overlaid on the terrain. This is the primary qualitative evidence of the swarm's emergent, decentralized coordination: with no communication and no shared plan, the four independently-trained agents visibly partition the grid into roughly separate operating regions, each drone's path clustering around the targets nearest to its own random start position.

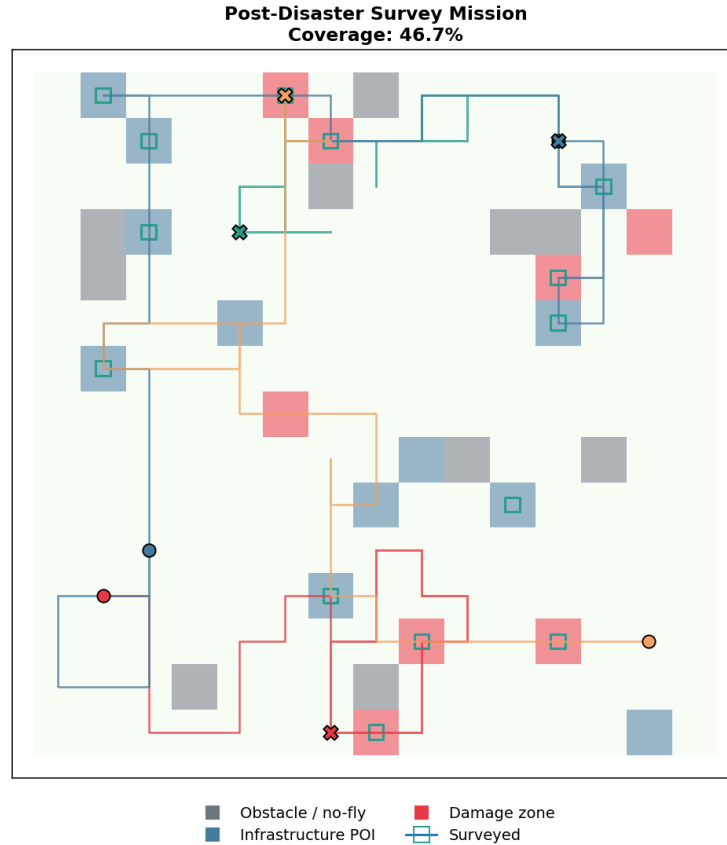


Figure 5. Mission map for the best-of-10 greedy evaluation run. Circles mark each drone's start position, X markers its final position; the outlined squares are cells the swarm collectively surveyed. Grey = obstacle, blue = infrastructure POI, red = damage zone.

7.4 Quantitative Survey Report

analytics.py additionally emits a plain-text survey_report.txt summarizing the same run in the format a post-mission analytics dashboard would present to an operator:

Metric	Result
Overall area coverage	46.7%
Infrastructure POIs mapped	8 / 12 (66.7%)
Damage zones surveyed	6 / 8 (75.0%)
Mission duration	300 timesteps (budget exhausted)
Best-of-10 greedy evaluation coverage	53.3%
Mean greedy evaluation coverage (20 runs)	≈ 35% (std ≈ 12%), up to 60% in individual runs

Two figures are reported for coverage because they answer different questions: the single best-of-10 run (used for the mission map above) shows what the swarm is capable of in a favorable episode, while

the 20-run mean and standard deviation shows the realistic, repeatable expected performance of the learned policy — the latter is the honest number for any downstream decision-making. The visibly higher survey rate for damage zones (75.0%) versus general infrastructure POIs (66.7%) is a direct, measurable consequence of the reward function's disaster-priority weighting (+15 vs. +10, Section 4.4) — the swarm has genuinely learned to prioritize disaster-critical terrain, not just to cover area indiscriminately.

8. Known Limitations & The Debugging Journey

This section documents a genuine convergence failure encountered during development and how it was diagnosed and fixed, on the view that an accurate account of what didn't work the first time is more useful documentation than a description that only shows the final, working state.

8.1 The Symptom

The very first working version of this framework (state = own grid-cell bucket + target direction, no shaping reward) trained with a reward curve that appeared to plateau, but the fully-greedy evaluation policy achieved only 6.7% coverage in a single test episode — dramatically worse than the ~50% coverage observed during training itself, when agents still had a small amount of exploration noise ($\epsilon = 0.05\text{--}0.2$).

8.2 First Hypothesis: Sparse Rewards

The first suspicion was that the +10/+15 survey rewards were too sparse for tabular Q-learning to propagate back to distant states quickly enough. Potential-based reward shaping (Section 4.5) was added, giving every step a dense directional reward proportional to progress toward the nearest un-mapped target. This measurably helped — average training coverage rose from the 50s into the low 60s (%) — but a 20-episode greedy evaluation still averaged only 14.8% coverage, with high variance (std 8.1%) and several episodes at or near 0%. Sparse rewards were a real contributing factor, but not the root cause of the evaluation collapse.

8.3 Correct Diagnosis: State Aliasing and Oscillation Cycles

Printing individual greedy-policy episode trajectories exposed the actual failure: agents were flying in short, repeating cycles — most commonly moving North then South then North then South indefinitely — never making net progress toward any target, despite each individual step's Q-value looking locally sensible.

The underlying cause is state aliasing: a state defined only by "target direction + nearby obstacles" cannot distinguish between two different physical situations that require opposite actions to make progress — for instance, a drone standing just north of its target and a drone standing just south of the same target, having each just arrived by moving toward it from the previous cell, can end up mapped to the exact same state, and the shared Q-value estimate for that state converges to whichever action was

reinforced more often in training, which is not always correct for both physical cases. Under greedy (arg-max) action selection at evaluation time, this can cause a genuine two-step deterministic cycle that exploration noise would previously have randomly broken out of — which explains why the problem was invisible during training ($\epsilon > 0$) and only surfaced at full greedy evaluation.

8.4 The Fix and Its Measured Effect

Adding the drone's previous action taken as a fourth state component (Section 4.3) breaks the aliasing: the same physical position now maps to a different state tuple depending on how the drone got there, giving the Q-learning process enough information to learn "don't immediately reverse the move you just made" as part of the policy. The before/after comparison, all on the same 20-run greedy evaluation protocol:

Configuration	Mean coverage (20 runs)	Notes
Sparse reward only, no shaping, no previous-action state	6.7%	single-episode spot check
+ potential-based shaping reward	14.8% (std 8.1%)	still exhibits oscillation cycles
+ previous-action added to state	$\approx 35\%$ (std 12%), up to 60% in best runs	final shipped configuration

This progression is preserved in the codebase's comments and in the README's "Known limitation" section deliberately, rather than only shipping the final numbers — the failure mode (state aliasing causing greedy-policy cycles) is a general lesson for tabular RL, not specific to this project, and is worth being able to recognize quickly in future work.

8.5 The Deeper, Still-Present Limitation: Non-Stationarity

Even with the fix above, mean coverage across evaluation runs ($\approx 35\%$) remains well below the best individual runs (up to 60%), and standard deviation remains high (≈ 12 percentage points). This residual variance is consistent with the Independent Q-Learning non-stationarity problem described in Section 4.2: each agent's optimal policy genuinely depends on what the other three agents happen to do, which is not something a fully independent tabular learner can model. This is not a bug to be fixed within the current architecture — it is the expected, well-documented ceiling of decentralized tabular Q-learning for this class of problem, and is the primary motivation for the extensions proposed in Section 9.

9. Future Extensions

The current framework is a complete, working demonstration of distributed multi-agent Q-learning applied to swarm coordination, but several concrete extensions would be the natural next steps toward a production-grade system:

- **Deep Q-Networks (DQN)** — replace each drone's tabular Q-table with a small shared-architecture neural network taking a richer, continuous state (exact position, vector to nearest target, local obstacle map) as input. Function approximation generalizes across nearby states instead of aliasing them, directly addressing the root cause identified in Section 8.3, and removes the need to hand-design a small discrete state space at all.
- **Centralized Training, Decentralized Execution (CTDE)** — train a value function conditioned on the joint state/action of all drones (addressing the non-stationarity of Section 8.5 directly), while still deploying independent per-drone policies at inference time — algorithms such as QMIX or MADDPG are the standard reference points here.
- **Explicit task-allocation layer** — add an auction- or market-based bidding mechanism on top of the learned policies so that target assignment between drones has hard non-overlap guarantees, rather than relying entirely on emergent behavior from the shared coverage map.
- **Real geospatial data** — replace the synthetic randomly-scattered grid in `environment.py` with a rasterized real-world data source — e.g. OpenStreetMap building footprints for infrastructure POIs, or an actual post-event damage-assessment raster for damage-zone priorities — the state/reward machinery is entirely agnostic to how POIs are sourced.
- **Continuous state and action spaces** — move from a discretized grid to continuous (x, y) coordinates and continuous heading/speed actions, which is necessary for any eventual transfer from simulation to real flight controllers.
- **Horizontal scaling of the training loop itself** — as noted in Section 5.3, because each agent's Q-table update only touches its own data, the per-timestep agent loop in `SwarmCoordinator.run_episode()` is embarrassingly parallel and could be distributed across processes or machines to train much larger swarms.

10. Appendix

10.1 Glossary of Terms

Term	Definition
Q-learning	A model-free RL algorithm that learns the expected future reward, $Q(s,a)$, of taking action a in state s .
Tabular RL	Representing $Q(s,a)$ as an explicit lookup table rather than a function approximator such as a neural network.
Independent Q-Learning (IQL)	A multi-agent RL setup where every agent runs single-agent Q-learning independently, unaware of other agents' internal state.
ϵ -greedy policy	Action-selection rule: explore with probability ϵ , otherwise exploit (pick the highest-value known action).
Non-stationarity	In multi-agent RL, the phenomenon where the environment's effective dynamics change for one agent as other agents learn and change their behavior.
State aliasing	When two distinct situations that require different optimal actions are mapped to the same state representation, degrading a tabular policy's quality.

Term	Definition
Potential-based reward shaping	Adding $\gamma\Phi(s') - \Phi(s)$ to the reward for a hand-designed potential function Φ , which provably preserves the optimal policy while making learning faster.
Coverage ratio	The fraction of mission-relevant cells (POIs + damage zones) that have been surveyed by any drone.
CTDE	Centralized Training, Decentralized Execution — a multi-agent RL paradigm that trains with joint information but deploys independent per-agent policies.

10.2 Hyperparameter Quick Reference

```

QLearningAgent defaults (q_agent.py):
    n_actions      = 5      # N, S, W, E, Hover
    alpha          = 0.15   # learning rate
    gamma          = 0.95   # discount factor
    epsilon        = 1.0    # initial exploration rate
    epsilon_min    = 0.05   # exploration floor
    epsilon_decay  = 0.995  # multiplicative decay per episode

SwarmGridEnv defaults (environment.py):
    size           = 15     # grid is size x size
    n_drones       = 4
    n_poi          = 12     # infrastructure points of interest
    n_obstacles    = 10
    n_damage       = 8      # disaster damage zones
    seed           = 42     # reproducible terrain generation

Reward constants (environment.py, SwarmGridEnv.step):
    step_cost      = -0.1
    boundary_penalty = -2.0
    obstacle_penalty = -3.0
    collision_penalty = -5.0 (per colliding drone)
    poi_survey_bonus = +10.0
    damage_survey_bonus = +15.0
    shaping_weight  = 0.5   (x distance closed to nearest target)

Episode termination (environment.py, SwarmGridEnv.step):
    done = (coverage >= 1.0) OR (steps_taken >= 300)

```

10.3 How to Run

Requirements: Python 3, numpy, matplotlib.

```

pip install numpy matplotlib

python3 train.py      # train only; prints metrics to console
python3 analytics.py  # train + generate all reporting artifacts:
                      #   training_diagnostics.png
                      #   mission_map.png
                      #   survey_report.txt

```

To scale the swarm to more drones, change `n_drones` in the call to `SwarmGridEnv` (or the `train()` function's default) — no other code changes are required, since every additional drone is simply one more independent `QLearningAgent` instance.